CI/CD Pipeline for a Calculator REST API

Detailed Project Report · CSE 816 Software Production Engineering, Mini-Project

Item	Detail
Submitted by	Mayank Tanwar
Roll number	IMT2023012
Course	CSE 816 Software Production Engineering, M.Tech (CSE), Term I 2026-27
Faculty	Prof. B. Thangaraju
Contact	mayanktanwar01234@gmail.com · GitHub: Mayanktan
Date	29 October 2026
Source code	https://github.com/Mayanktan/calculator-devops
Docker image	https://hub.docker.com/r/mayanktan/calculator-devops
Stack	Java 21, Spring Boot 3.3.4, Maven, JUnit 5, Git/GitHub, Jenkins, Docker, Docker Compose, Nginx

Abstract

This report describes a small Calculator REST API built with Spring Boot and the automated DevOps pipeline around it. A push to the `main` branch on GitHub is detected by Jenkins, which compiles the code, runs 16 JUnit 5 tests, builds a multi-stage Docker image, pushes it to Docker Hub with credentials from the Jenkins credentials store, deploys it with Docker Compose behind an Nginx reverse proxy, checks the health endpoint and emails the result. Five real pipeline runs are reported, including an automatic deployment of a code change and a deliberately failing test that stopped the pipeline before anything was published. The report also records seven problems found while building the system, how each was diagnosed, and how it was fixed.

Contents

1. Introduction . . . 2
2. System Architecture . . . 2
3. The Application . . . 4
4. Testing . . . 6
5. Version Control with Git and GitHub . . . 8
6. Containerization with Docker . . . 9
7. Deployment with Docker Compose . . . 10
8. CI/CD Pipeline with Jenkins . . . 12
9. Results and Verification . . . 15
10. Challenges and Solutions . . . 17
11. How to Run and Reproduce . . . 18
12. Conclusion and Future Work . . . 18

Appendix A. Screenshots . . . 19

Appendix B. Versions . . . 23

1. Introduction

1.1 Problem statement

The assignment is to build a production-style Calculator REST API in Java with Spring Boot and to wrap it in a complete, automated pipeline using only the tools from the first three course modules: Git and GitHub for version control, Jenkins for continuous integration and delivery, Docker for containerization, and Docker Compose for deployment. The marks are for the engineering around the application, not for the maths: a code change pushed to GitHub has to be built, tested, packaged, containerized and deployed without manual steps.

1.2 Objectives

- Use Git and GitHub with feature branches, pull requests and a meaningful commit history.
- Write a declarative Jenkins pipeline covering build, test, package, image build, push and deploy.
- Write a clean multi-stage Dockerfile that follows efficiency and security best practices.
- Run a multi-container deployment with Docker Compose, with health checks and a reverse proxy.
- Connect everything into one trigger-driven workflow that also reports its result by email.

1.3 Scope

The project stops at Compose-based deployment on a single machine. Configuration management (Ansible), orchestration (Kubernetes) and monitoring (Prometheus and Grafana) belong to the end-of-semester capstone, which extends this same application; section 12 lists what is already in place for that.

1.4 Deliverables

Deliverable	Where
Source code, Jenkinsfile, Dockerfile, docker-compose.yml	https://github.com/Mayanktan/calculator-devops
Public Docker image	https://hub.docker.com/r/mayanktan/calculator-devops
Short report (2-3 pages, as required)	`docs/report.pdf`
This detailed report	`detailed-report.pdf`
Screenshots: green pipeline run, emails, docker ps, API responses	`docs/screenshots/` and Appendix A
Run and test guide	`test.pdf`

2. System Architecture

2.1 Pipeline and deployment

Figure 1 shows the delivery path from a commit to a running container, and Figure 2 shows what is running once a build has been deployed. Jenkins runs directly on the same Mac as Docker, so the Deploy stage can manage the Compose stack and check it on `http://localhost`.

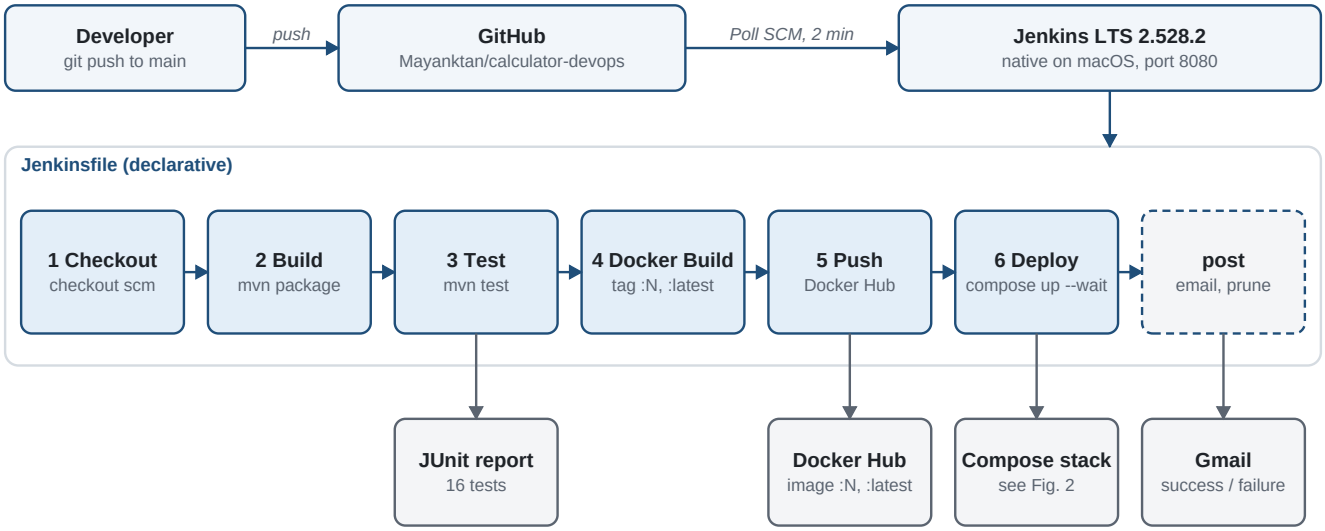


Figure 1. Pipeline: a push to GitHub is picked up by Jenkins, which runs six stages; the side outputs are the test report, the pushed image, the deployed stack and the email.

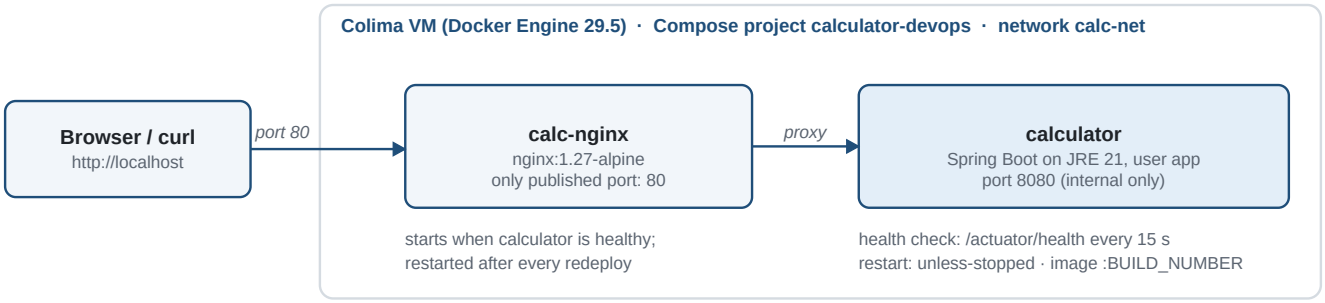


Figure 2. Deployment: only Nginx is reachable from the host; the application's port 8080 stays inside the `calc-net` network.

2.2 Components

Component	Technology	Role
Application	Spring Boot 3.3.4 on Java 21	REST endpoints, validation, Actuator health endpoint
Build tool	Maven 3.9, Surefire 3.2.5	Compile, test, package the executable JAR, JUnit XML reports
Source control	Git, GitHub	History, branches, pull requests; source for every Jenkins build
CI/CD server	Jenkins LTS 2.528.2	Polls GitHub and runs the declarative pipeline
Container runtime	Docker Engine 29.5 in Colima 0.10	Builds and runs images on the Mac
Registry	Docker Hub	Stores `mayanktan/calculator-devops:<build>` and `:latest`
Orchestration	Docker Compose 5.6	Starts the app and the proxy with one command
Reverse proxy	Nginx 1.27 (alpine)	Single public entry point on port 80
Notification	Email Extension plugin, Gmail SMTP	Success and failure emails

2.3 Repository structure

Repository layout

```
calculator-devops/
|-- src/main/java/com/example/calculator/
|   |-- CalculatorApplication.java        Spring Boot entry point
|   |-- controller/CalculatorController.java
|   |-- service/CalculatorService.java
|   |-- dto/CalculationResponse.java, ApiError.java
|   `-- exception/DivideByZeroException.java, GlobalExceptionHandler.java
|-- src/main/resources/application.properties
|-- src/test/java/com/example/calculator/
|   |-- CalculatorServiceTest.java        8 unit tests
|   `-- CalculatorControllerIT.java       8 MockMvc integration tests
|-- pom.xml                               Maven build
|-- Dockerfile, .dockerignore             multi-stage image
|-- docker-compose.yml, nginx/nginx.conf  app + reverse proxy
|-- Jenkinsfile                           declarative pipeline
|-- scripts/preflight.sh                  pre-demo checks
|-- README.md, test.pdf, detailed-report.pdf
`-- docs/report.pdf, docs/screenshots/
```

2.4 Environment

Item	Version or setting
Machine	MacBook, Apple Silicon (arm64), 8 GB RAM, macOS 26.5
Java	JDK 21 (Homebrew `openjdk@21`) for Maven; Eclipse Temurin 21 inside the images
Maven	3.9.11 on the host; 3.9.9 in the build image
Docker	Colima 0.10.3 VM (4 CPU, 4 GB), Docker Engine 29.5.2, CLI 29.9.2, Compose 5.6.1, Buildx 0.38
Jenkins	LTS 2.528.2 installed with Homebrew, running as the user on 127.0.0.1:8080

3. The Application

3.1 REST API

Method and path	Example	Response
`GET /api/v1/add?a=&b=`	`/api/v1/add?a=2&b=3`	200, result 5.0
`GET /api/v1/subtract?a=&b=`	`/api/v1/subtract?a=5&b=2`	200, result 3.0
`GET /api/v1/multiply?a=&b=`	`/api/v1/multiply?a=4&b=3`	200, result 12.0
`GET /api/v1/divide?a=&b=`	`/api/v1/divide?a=10&b=4`	200, result 2.5; b=0 gives 400
`GET /api/v1/power?a=&b=`	`/api/v1/power?a=2&b=10`	200, result 1024.0
`GET /actuator/health`	`/actuator/health`	200 {"status":"UP", ...} (DOWN gives 503)

Every success response has the shape the specification asks for, and every client error returns HTTP 400 with a structured body:

Success and error bodies (real responses)

```
{"operation":"add","a":2.0,"b":3.0,"result":5.0}

{"timestamp":"2026-10-28T18:38:33.290455648Z","status":400,"error":"Bad Request",
 "message":"Cannot divide by zero","path":"/api/v1/divide"}
```

3.2 Design

The code follows the usual Spring layering. The controller only reads the query parameters and wraps results; the arithmetic lives in a service with no web code, so it can be unit-tested without Spring; and one `@RestControllerAdvice` turns every client error into the same JSON shape. Both response types are Java records, which Jackson serialises directly.

service/CalculatorService.java: divide and power (lines 21-30)

```
    public double divide(double a, double b) {
        if (b == 0.0) {
            throw new DivideByZeroException("Cannot divide by zero");
        }
        return a / b;
    }

    public double power(double a, double b) {
        return Math.pow(a, b);
    }
```

controller/CalculatorController.java: one endpoint (lines 23-26)

```
    @GetMapping("/add")
    public CalculationResponse add(@RequestParam double a, @RequestParam double b) {
        return new CalculationResponse("add", a, b, service.add(a, b));
    }
```

exception/GlobalExceptionHandler.java (lines 17-49)

```
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(DivideByZeroException.class)
    public ResponseEntity<ApiError> handleDivideByZero(DivideByZeroException ex, HttpServletRequest
            req) {
        return badRequest(ex.getMessage(), req);
    }

    // e.g. /api/v1/add?a=2 (no b)
    @ExceptionHandler(MissingServletRequestParameterException.class)
    public ResponseEntity<ApiError> handleMissingParam(MissingServletRequestParameterException ex,
                                                       HttpServletRequest req) {
        return badRequest("Missing required parameter: " + ex.getParameterName(), req);
    }

    // e.g. /api/v1/add?a=foo&b=3
    @ExceptionHandler(MethodArgumentTypeMismatchException.class)
    public ResponseEntity<ApiError> handleTypeMismatch(MethodArgumentTypeMismatchException ex,
                                                       HttpServletRequest req) {
        return badRequest("Invalid value for parameter '" + ex.getName() + "': must be a number", req);
    }

    private ResponseEntity<ApiError> badRequest(String message, HttpServletRequest req) {
        HttpStatus status = HttpStatus.BAD_REQUEST;
        ApiError body = new ApiError(
                Instant.now(),
                status.value(),
                status.getReasonPhrase(),
                message,
                req.getRequestURI());
        return ResponseEntity.status(status).body(body);
    }
}
```

3.3 Validation rules

Input	Status	Message in the error body
`b=0` on divide	400	Cannot divide by zero
missing parameter, e.g. `/add?a=2`	400	Missing required parameter: b
non-numeric value, e.g. `a=foo`	400	Invalid value for parameter 'a': must be a number
empty value, e.g. `a=&b=3`	400	Invalid value for parameter 'a': must be a number
unknown path, e.g. `/api/v1/sqrt`	404	Spring Boot's default Not Found body

3.4 Configuration

src/main/resources/application.properties

```
# Application identity
spring.application.name=calculator-devops
server.port=8080

# Expose Actuator health endpoint (spec requirement)
management.endpoints.web.exposure.include=health,info
management.endpoint.health.show-details=always
management.endpoint.health.probes.enabled=true

# Error output
server.error.include-message=always
server.error.include-stacktrace=never
```

Only the `health` and `info` Actuator endpoints are exposed. Health details are shown to make the demo clearer; in a real production system `show-details` would be `never` or `when-authorized`. The liveness and readiness probes are already enabled for the Kubernetes work in the capstone.

4. Testing

4.1 Strategy

There are two layers of JUnit 5 tests. Unit tests create `CalculatorService` directly, with no Spring context, and run in milliseconds. Integration tests start the whole application with `@SpringBootTest` and call the real endpoints through `MockMvc`, checking HTTP status codes and JSON fields, including the error bodies and the health endpoint.

4.2 Test inventory

Class	Test method	What it checks
CalculatorServiceTest (unit)	`addBasic`	add — basic
CalculatorServiceTest (unit)	`addNegatives`	add — negatives
CalculatorServiceTest (unit)	`subtractBasic`	subtract — basic
CalculatorServiceTest (unit)	`multiplyBasic`	multiply — basic
CalculatorServiceTest (unit)	`multiplyByZero`	multiply — by zero
CalculatorServiceTest (unit)	`divideBasic`	divide — basic
CalculatorServiceTest (unit)	`divideByZeroThrows`	divide — by zero throws DivideByZeroException
CalculatorServiceTest (unit)	`powerBasic`	power — basic
CalculatorControllerIT (MockMvc)	`addEndpoint`	GET /api/v1/add returns 200 + correct JSON
CalculatorControllerIT (MockMvc)	`subtractEndpoint`	GET /api/v1/subtract returns 200 + correct difference
CalculatorControllerIT (MockMvc)	`multiplyEndpoint`	GET /api/v1/multiply returns 200 + correct product
CalculatorControllerIT (MockMvc)	`divideEndpoint`	GET /api/v1/divide returns 200 + correct quotient

Class	Test method	What it checks
CalculatorControllerIT (MockMvc)	`divideByZeroReturns400`	GET /api/v1/divide with b=0 returns 400 + error JSON
CalculatorControllerIT (MockMvc)	`missingParamReturns400`	GET /api/v1/add with missing param returns 400
CalculatorControllerIT (MockMvc)	`invalidParamReturns400`	GET /api/v1/add with non-numeric param returns 400
CalculatorControllerIT (MockMvc)	`healthEndpoint`	GET /actuator/health returns UP

4.3 Examples

CalculatorServiceTest.java: the divide-by-zero unit test (lines 57-64)

```
    @Test
    @DisplayName("divide — by zero throws DivideByZeroException")
    void divideByZeroThrows() {
        DivideByZeroException ex = assertThrows(
                DivideByZeroException.class,
                () -> service.divide(10, 0));
        assertEquals("Cannot divide by zero", ex.getMessage());
    }
```

CalculatorControllerIT.java: the divide-by-zero integration test (lines 58-67)

```
    @Test
    @DisplayName("GET /api/v1/divide with b=0 returns 400 + error JSON")
    void divideByZeroReturns400() throws Exception {
        mockMvc.perform(get("/api/v1/divide").param("a", "10").param("b", "0"))
                .andExpect(status().isBadRequest())
                .andExpect(jsonPath("$.status").value(400))
                .andExpect(jsonPath("$.error").value("Bad Request"))
                .andExpect(jsonPath("$.message").value("Cannot divide by zero"))
                .andExpect(jsonPath("$.path").value("/api/v1/divide"));
    }
```

4.4 Running the tests and reports

`mvn test` runs all 16 tests and writes JUnit XML files to `target/surefire-reports/`, which the Jenkins Test stage publishes. Surefire only picks up `*Test.java` files by default, so the integration test class would silently never run; the build declares both patterns explicitly:

pom.xml: Surefire configuration (lines 60-72)

```
            <!-- Runs JUnit 5 tests and writes target/surefire-reports/*.xml.
                 Surefire skips *IT.java by default, so include it explicitly. -->
            <plugin>
                <groupId>org.apache.maven.plugins</groupId>
                <artifactId>maven-surefire-plugin</artifactId>
                <version>3.2.5</version>
                <configuration>
                    <includes>
                        <include>**/*Test.java</include>
                        <include>**/*IT.java</include>
                    </includes>
                </configuration>
            </plugin>
```

> Result: 16 tests, 0 failures in every green Jenkins build; build #4 shows the expected 15/16 when a test was broken on purpose (section 9.4).

5. Version Control with Git and GitHub

5.1 Practices

- **Public repository** `Mayanktan/calculator-devops` with a README that covers building, testing, running and calling the API, and the pipeline setup.
- **Conventional Commits:** every message starts with `feat:`, `fix:`, `test:`, `build:`, `ci:`, `docs:` or `chore:`, so the log reads as a list of changes.
- **.gitignore first:** the first commit adds only `.gitignore` (Maven `target/`, IDE folders, OS files, logs, `.env`, `*.pem`, `*.key`), so build output was never tracked.
- **No secrets in the repository:** the Docker Hub token and the Gmail App Password exist only in the Jenkins credentials store; the Jenkinsfile refers to them by ID.

5.2 Branches and pull requests

PR	Branch	Contents
#1	`feat/calculator-endpoints`	Service, DTOs, controller, exception handler and the 16 tests (5 commits)
#2	`feat/dockerfile`	Multi-stage Dockerfile and .dockerignore
#3	`feat/docker-compose`	Compose stack and Nginx configuration
#4	`ci/jenkinsfile`	Declarative Jenkins pipeline
#5	`docs/readme`	README, preflight script and the run and test guide (3 commits)

All five pull requests were reviewed and merged on GitHub by the repository owner. After the merges, three more commits were pushed to `main` to test the pipeline (sections 9.3 to 9.5). Up to the pipeline tests, `main` has 22 commits:

git log --reverse (hash, author date and time, message)

```
3f87b81  28 Oct 23:27  chore: project .gitignore
4038b66  28 Oct 23:27  build: Spring Boot 3.3 pom.xml with web, actuator, validation and test
b8239de  28 Oct 23:27  feat: Spring Boot application class and actuator config
7e9d290  28 Oct 23:29  feat: calculator service with five operations and divide-by-zero exception
902312b  28 Oct 23:29  feat: CalculationResponse and ApiError DTO records
e50fcbf  28 Oct 23:29  feat: REST controller for /api/v1/{add,subtract,multiply,divide,power}
0d064d4  28 Oct 23:29  feat: global exception handler returns 400 with ApiError JSON
ee4871c  28 Oct 23:29  test: JUnit 5 service unit tests and MockMvc integration tests
ab24b48  28 Oct 23:29  feat: multi-stage Dockerfile with non-root user and .dockerignore
e3259c9  28 Oct 23:29  feat: compose stack with nginx reverse proxy and health check
399530f  28 Oct 23:30  ci: declarative pipeline with six stages, Docker Hub push and email
c64d8bc  28 Oct 23:30  docs: README with build, run, pipeline and Jenkins setup
32338e0  28 Oct 23:30  chore: pre-demo preflight script
7b63217  28 Oct 23:30  docs: run and test guide (test.pdf)
75ccd0c  30 Oct 20:53  Merge pull request #1 from Mayanktan/feat/calculator-endpoints
feba814  30 Oct 20:53  Merge pull request #2 from Mayanktan/feat/dockerfile
d97e18f  30 Oct 20:53  Merge pull request #3 from Mayanktan/feat/docker-compose
95cfe7e  30 Oct 20:54  Merge pull request #4 from Mayanktan/ci/jenkinsfile
5ac3889  30 Oct 20:54  Merge pull request #5 from Mayanktan/docs/readme
6e870a3  28 Oct 23:45  feat: list the endpoints on the landing page
43aa856  28 Oct 23:48  test: deliberately break a test to check the pipeline stops
e03f008  28 Oct 23:49  Revert "test: deliberately break a test to check the pipeline stops"
```

6. Containerization with Docker

6.1 Dockerfile

Dockerfile

```
# ============================================================
# Stage 1 — BUILD: compile the JAR using Maven + JDK 21
# ============================================================
FROM maven:3.9.9-eclipse-temurin-21 AS build
WORKDIR /workspace

# Copy only pom.xml first so the dependency layer is cached
# until pom.xml changes.
COPY pom.xml .
RUN mvn -B -ntp dependency:go-offline

# Source changes only invalidate the layers from here down.
COPY src ./src
RUN mvn -B -ntp clean package -DskipTests

# ============================================================
# Stage 2 — RUNTIME: slim JRE, just enough to run the JAR
# ============================================================
FROM eclipse-temurin:21-jre-alpine

# Non-root user and group
RUN addgroup -S app && adduser -S app -G app
WORKDIR /app

# Copy ONLY the built JAR out of the build stage
COPY --from=build --chown=app:app /workspace/target/calculator-devops-*.jar /app/app.jar

EXPOSE 8080
USER app

# Exec form: Java runs as PID 1, so 'docker stop' signals reach it.
# JVM flags come from JAVA_TOOL_OPTIONS (set in docker-compose.yml).
ENTRYPOINT ["java", "-jar", "/app/app.jar"]
```

6.2 Best practices applied

Requirement	How the Dockerfile meets it
Multi-stage build	`maven:3.9.9-eclipse-temurin-21` builds the JAR; `eclipse-temurin:21-jre-alpine` runs it
Runtime copies only the JAR	`COPY --from=build ... calculator-devops-*.jar /app/app.jar`, nothing else
Minimal base image	Alpine JRE instead of a full JDK and Maven image
Non-root user	`addgroup`/`adduser` create `app`; `USER app` before the entry point; `id` shows uid 100
Explicit EXPOSE	`EXPOSE 8080`
Dependency layer caching	`pom.xml` is copied and `dependency:go-offline` runs before the sources are copied
.dockerignore	Excludes `target/`, `.git/`, IDE folders, docs, scripts and secrets from the build context
Signals and JVM flags	Exec-form `ENTRYPOINT`, so Java is PID 1; heap flags come from `JAVA_TOOL_OPTIONS`

6.3 Verification

- The image builds from an empty cache and is **331 MB** locally, **90.56 MB** compressed on Docker Hub (`linux/arm64`).
- One `docker run -p 8090:8080` answers on the mapped port; `docker exec calc id` prints `uid=100(app) gid=101(app)`.
- Re-builds reuse the cached dependency layer: the Docker Build stage takes about 2-3 s in Jenkins.

7. Deployment with Docker Compose

7.1 Compose file

docker-compose.yml

```
# Fixed project name so local runs and Jenkins runs manage the same stack.
name: calculator-devops

services:
  calculator:
    # Jenkins injects the build-tagged image; locally it builds from the Dockerfile.
    image: ${DOCKER_IMAGE:-mayanktan/calculator-devops:latest}
    build: .
    container_name: calculator
    environment:
      - SPRING_PROFILES_ACTIVE=prod
      - JAVA_TOOL_OPTIONS=-Xms128m -Xmx384m   # read by the JVM automatically
    expose:
      - "8080"                # internal only; nginx reaches it
    networks:
      - calc-net
    healthcheck:
      test: ["CMD", "wget", "-qO-", "http://localhost:8080/actuator/health"]
      interval: 15s
      timeout: 5s
      retries: 5
      start_period: 20s
    restart: unless-stopped

  nginx:
    image: nginx:1.27-alpine
    container_name: calc-nginx
    depends_on:
      calculator:
        condition: service_healthy
        restart: true         # restart nginx after calculator is redeployed (new IP, fresh config)
    ports:
      - "80:80"               # public entry point
    volumes:
      - ./nginx/nginx.conf:/etc/nginx/conf.d/default.conf:ro
    networks:
      - calc-net
    restart: unless-stopped

networks:
  calc-net:
    driver: bridge
```

7.2 Nginx reverse proxy

nginx/nginx.conf

```
# No underscore in the name: it is sent as the Host header otherwise, and Tomcat rejects it
upstream calc-backend {
    server calculator:8080;   # 'calculator' is the service name in compose
}

server {
    listen 80 default_server;
    server_name _;

    # Inherited by every proxied location below
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;

    # Pass the API through
    location /api/ {
        proxy_pass http://calc-backend;
    }

    # Health endpoint for external monitoring
    location /actuator/ {
        proxy_pass http://calc-backend;
    }

    # Small landing page
    location / {
        default_type text/html;
        return 200 '<h1>Calculator DevOps</h1><p>Endpoints: /api/v1/add, /subtract, /multiply, /divide,
            /power (?a=&b=) and /actuator/health</p><p>Try <a
            href="/api/v1/add?a=2&b=3">/api/v1/add?a=2&b=3</a></p>';
    }
}
```

7.3 How the requirements are met

Requirement	Implementation
Brings up the calculator service	Service `calculator`, image injected by Jenkins through `DOCKER_IMAGE`
A defined network	User-defined bridge network `calc-net`; services find each other by name
Health check on /actuator/health	`wget` every 15 s, 5 retries, 20 s start period
Restart policy	`restart: unless-stopped` on both services
Environment variables	`SPRING_PROFILES_ACTIVE` and `JAVA_TOOL_OPTIONS` (JVM heap)
Second component	Nginx reverse proxy; the only published port is 80
One command starts everything	`docker compose up -d` (Jenkins adds `--wait` and `--remove-orphans`)

7.4 Behaviour checked on the real stack

- **Start order:** Compose waits for `calculator` to report Healthy before starting `calc-nginx`.
- **Redeploy:** with a new image tag, Compose replaces the app container and then restarts Nginx because of `depends_on: restart: true`, so the proxy picks up the new container and any `nginx.conf` change.
- **Idempotent:** running the same `docker compose up -d` again changes nothing; both containers stay up.
- **Isolation:** `docker compose ps` shows only `8080/tcp` for the app and `0.0.0.0:80->80/tcp` for Nginx.

8. CI/CD Pipeline with Jenkins

8.1 Jenkins setup

Item	Setting
Installation	`brew install jenkins-lts`, started with `brew services`; listens on 127.0.0.1:8080
Plugins	Pipeline, Git, GitHub, Credentials Binding, JUnit, Email Extension, Timestamper, Pipeline Graph View (versions in Appendix B)
Global environment	`JAVA_HOME=/opt/homebrew/opt/openjdk@21` and `PATH+EXTRA=/opt/homebrew/bin:/usr/local/bin`
Credentials	`dockerhub-creds` (Docker Hub user and access token) and `gmail-smtp` (Gmail App Password)
Email	Extended E-mail Notification: `smtp.gmail.com`, port 465, SSL, HTML content, sender address set
Job	Pipeline `calculator-devops`: script from SCM, Git URL of the repo, branch `*/main`, `Jenkinsfile`
Triggers	Poll SCM `H/2 * * * *`; the GitHub hook trigger is also enabled for a future webhook

8.2 The Jenkinsfile

The pipeline is declarative. The environment block defines the image name once; options turn off the implicit checkout, allow only one build at a time, add timestamps, cap a build at 20 minutes and keep 15 builds.

Jenkinsfile: environment and options (lines 1-20)

```
pipeline {
    agent any

    environment {
        // Change these two values to your own
        DOCKERHUB_USER = 'mayanktan'
        NOTIFY_EMAIL   = 'mayanktanwar01234@gmail.com'

        IMAGE_NAME = "${DOCKERHUB_USER}/calculator-devops"
        IMAGE_TAG  = "${env.BUILD_NUMBER}"
        FULL_IMAGE = "${IMAGE_NAME}:${IMAGE_TAG}"
    }

    options {
        skipDefaultCheckout()       // the Checkout stage below does it
        disableConcurrentBuilds()   // one deploy at a time
        timestamps()
        timeout(time: 20, unit: 'MINUTES')
        buildDiscarder(logRotator(numToKeepStr: '15'))
    }
```

Jenkinsfile: the six stages (lines 22-95)

```
    stages {
        stage('Checkout') {
            steps {
                checkout scm
                sh 'git log -1 --oneline'
            }
        }

        stage('Build') {
            steps {
                sh 'mvn -B -ntp clean package -DskipTests'
            }
        }

        stage('Test') {
            steps {
                sh 'mvn -B -ntp test'
            }
            post {
                always {
                    // Publish results even when tests fail
                    junit testResults: 'target/surefire-reports/*.xml', allowEmptyResults: false
                }
            }
        }

        stage('Docker Build') {
            steps {
                sh "docker build -t ${FULL_IMAGE} -t ${IMAGE_NAME}:latest ."
                sh "docker images ${IMAGE_NAME}"
            }
        }

        stage('Push') {
            steps {
                withCredentials([usernamePassword(
                        credentialsId: 'dockerhub-creds',
                        usernameVariable: 'DH_USER',
                        passwordVariable: 'DH_PASS')]) {
                    sh '''
                        echo "$DH_PASS" | docker login -u "$DH_USER" --password-stdin
                        docker push "$IMAGE_NAME:$IMAGE_TAG"
                        docker push "$IMAGE_NAME:latest"
                        docker logout
                    '''
                }
            }
        }

        stage('Deploy') {
            steps {
                // 'up -d' on a running stack only recreates what changed,
                // so re-runs are safe. '--wait' blocks until healthy.
                sh '''
                    export DOCKER_IMAGE="$IMAGE_NAME:$IMAGE_TAG"
                    echo "Deploying $DOCKER_IMAGE ..."
                    docker compose up -d --remove-orphans --wait
                    docker compose ps

                    # Verify health through the nginx proxy
                    for i in $(seq 1 12); do
                        if curl -fsS http://localhost/actuator/health | grep -q '"status":"UP"'; then
                            echo "Healthy after $i tries"
                            exit 0
                        fi
                        sleep 5
                    done
                    echo "Not healthy after 1 min"
                    docker compose logs
                    exit 1
                '''
            }
```

```
        }
    }
```

Jenkinsfile: post actions (lines 97-134)

```
    post {
        success {
            emailext(
                subject: "[Jenkins] SUCCESS: ${JOB_NAME} #${BUILD_NUMBER}",
                body: """
                    <h2 style="color:#2E8B57">Build succeeded</h2>
                    <ul>
                        <li><b>Job:</b> ${JOB_NAME}</li>
                        <li><b>Build:</b> #${BUILD_NUMBER}</li>
                        <li><b>Image:</b> ${FULL_IMAGE}</li>
                        <li><b>URL:</b> <a href="${BUILD_URL}">${BUILD_URL}</a></li>
                    </ul>
                """,
                to: "${NOTIFY_EMAIL}",
                mimeType: 'text/html'
            )
        }
        failure {
            emailext(
                subject: "[Jenkins] FAILED: ${JOB_NAME} #${BUILD_NUMBER}",
                body: """
                    <h2 style="color:#B23A48">Build failed</h2>
                    <ul>
                        <li><b>Job:</b> ${JOB_NAME}</li>
                        <li><b>Build:</b> #${BUILD_NUMBER}</li>
                        <li><b>Console:</b> <a href="${BUILD_URL}console">${BUILD_URL}console</a></li>
                    </ul>
                """,
                to: "${NOTIFY_EMAIL}",
                mimeType: 'text/html'
            )
        }
        always {
            // Remove old images so the agent's disk doesn't fill up
            sh 'docker image prune -f --filter "until=72h" || true'
        }
    }
}
```

8.3 Stages

Stage	What it does	Fails the build when
Checkout	`checkout scm` from `main`, then prints the commit	GitHub cannot be reached
Build	`mvn -B -ntp clean package -DskipTests`	the code does not compile
Test	`mvn test`; `junit` publishes the XML reports, even when tests fail	any test fails
Docker Build	Builds the image tagged with `BUILD_NUMBER` and `latest`	the image cannot be built
Push	Logs in with `dockerhub-creds` (`--password-stdin`), pushes both tags, logs out	login or push is rejected
Deploy	`docker compose up -d --remove-orphans --wait`, then up to 12 health checks through Nginx	the app is not healthy within a minute
post	Success or failure email; prunes dangling images older than 72 hours	never

8.4 Design decisions

- **Credentials stay in Jenkins.** `withCredentials` exposes the Docker Hub token only inside the Push step, masked in the log, and `--password-stdin` keeps it out of the process list.
- **Re-runnable and idempotent.** Each build pushes a unique tag; `compose up -d` only recreates what changed; the fixed Compose project name means Jenkins and manual runs manage the same stack; `disableConcurrentBuilds()` stops two deploys from overlapping. Nothing is torn down before deploying.
- **Fail fast.** A failing stage skips every later stage, so a broken build is never pushed or deployed.
- **Trigger.** Poll SCM every 2 minutes, which the specification accepts. A webhook needs GitHub to reach Jenkins from the internet; this Jenkins has no login and listens only on localhost, so exposing it through a tunnel would be unsafe. The GitHub hook trigger is already switched on for when Jenkins is secured.

9. Results and Verification

9.1 Build history

Build	Started	Triggered by	Result	Duration	Tests
#1	23:41:51	first run from the command line	SUCCESS	180 s	16 / 16
#2	23:44:52	Build Now on the web page	SUCCESS	37 s	16 / 16
#3	23:47:08	SCM change: landing page commit	SUCCESS	40 s	16 / 16
#4	23:49:09	SCM change: deliberately broken test	FAILURE	17 s	15 / 16
#5	23:51:10	SCM change: revert	SUCCESS	40 s	16 / 16

Each build sent its notification email, and all five arrived in the Gmail inbox: four with the subject "[Jenkins] SUCCESS: calculator-devops #N" and one "[Jenkins] FAILED: calculator-devops #4" (Figure A2 and Figure A3).

9.2 Stage timings (seconds)

Build	Checkout	Build	Test	Docker Build	Push	Deploy	Post
#1	1.6	2.9	3.4	3.2	151.9	7.0	6.2
#2	1.2	2.7	3.3	2.7	13.1	7.1	5.4
#3	1.7	3.0	3.3	2.7	14.0	7.1	5.8
#4	1.6	2.5	3.4 (failed)	skipped	skipped	skipped	6.9
#5	1.8	2.6	4.4	2.4	12.7	7.4	5.9

The first Push took 152 s because every image layer was uploaded; later pushes only send the changed layer. Docker Build stays at 2-3 s because the dependency layer is cached.

9.3 Automatic deployment of a change (build #3)

- 23:45:51: the commit "feat: list the endpoints on the landing page" was pushed to `main`; no Jenkins action.
- 23:47:08: Poll SCM found the new commit and started build #3, 77 s after the push.
- 23:47:48: build #3 finished green; Compose deployed image `:3`, Nginx was restarted and `http://localhost/` showed the new text.

9.4 A failing test stops the pipeline (build #4)

Build #4 failed on purpose. The specification requires the pipeline to "fail the build on test failure", so this was tested with a real failing test rather than assumed.

- 23:48:12: a commit changed one assertion in `CalculatorServiceTest.addBasic` from `assertEquals(5.0, service.add(2, 3), 1e-9)` to expect `6.0`, and was pushed. The application code was not changed; 2 + 3 still correctly returns 5.

- 23:49:09: build #4 started on its own; the Test stage failed with `CalculatorServiceTest.addBasic: expected: <6.0> but was: <5.0>` and Jenkins recorded 15 of 16 tests passing.
- Docker Build, Push and Deploy were reported as "skipped due to earlier failure(s)"; no tag 4 exists on Docker Hub, and the running application stayed on image `:3`.
- The post section sent the "[Jenkins] FAILED: calculator-devops #4" email, which arrived (Figure A3).

9.5 Recovery (build #5)

A revert commit restored the assertion. Build #5 started automatically at 23:51:10 and finished green in 40 s, deploying image `:5`, which is also `latest` on Docker Hub.

9.6 The deployed system

Real output (compose ps trimmed to the main columns; preflight output joined onto fewer lines)

```
$ docker compose ps
NAME         IMAGE                           STATUS                    PORTS
calc-nginx   nginx:1.27-alpine               Up 16 minutes             0.0.0.0:80->80/tcp
calculator   mayanktan/calculator-devops:5   Up 16 minutes (healthy)   8080/tcp

$ curl -s 'http://localhost/api/v1/power?a=2&b=10'
{"operation":"power","a":2.0,"b":10.0,"result":1024.0}

$ ./scripts/preflight.sh
=== 1. docker daemon ===  OK      === 2. jenkins up ===  OK      === 3. ngrok tunnel ===  none
=== 4. compose stack healthy ===  OK      === 5. GitHub repo reachable ===  OK
=== 6. Docker Hub repo exists ===  OK
All preflight checks passed — ready to demo.
```

9.7 Requirements traceability

Requirement	Evidence
Five endpoints, JSON format, HTTP 400 errors, Actuator	Section 3; Figure A5
JUnit 5 for every operation, divide by zero, MockMvc test	Section 4.2; Figure A6, Figure A8
mvn test produces JUnit XML for Jenkins	Test stage `junit` step; test trend in Figure A6
Public repo, README, .gitignore, no secrets	Section 5.1
Meaningful history, feature branches, merged PRs	Section 5.2; Figure A10, Figure A11
Declarative Jenkinsfile with the six stages	Section 8.2; Figure A1
Automatic trigger from GitHub	Builds #3 to #5 "Started by an SCM change" (section 9.1)
Fails on test failure	Build #4 (section 9.4); Figure A7
Image tagged with the build number, pushed with Jenkins credentials	Sections 8.3, 8.4; Figure A9
Deploy with compose up -d and verify health	Deploy stage; section 9.6; Figure A4
Email on success and failure	post section; Figure A2, Figure A3
Idempotent re-runs	Builds #2 and #3 redeployed over a running stack (section 8.4)
Multi-stage, non-root, EXPOSE, caching, .dockerignore	Section 6.2
Network, health check, restart policy, env vars, second component	Section 7.3

10. Challenges and Solutions

10.1 Health check through Nginx returned HTTP 400

Symptom	Cause	Fix and verification
In the first Compose test, the API worked through Nginx but `/actuator/health` returned Tomcat's HTML "HTTP Status 400" page. The Deploy stage checks exactly this URL, so every deployment would have failed.	The `/api/` location set `proxy_set_header Host $host`, but `/actuator/` did not, so Nginx sent the upstream name `calc_backend` as the Host header. Tomcat rejects host names containing an underscore.	Renamed the upstream to `calc-backend` and moved the proxy headers to the `server` block so every location sends them. All routes returned 200; build #1's health check passed on the first try.

10.2 Integration tests never ran

Symptom	Cause	Fix and verification
`mvn test` reported 8 tests instead of 16.	Surefire's default includes match `*Test.java`, `Test*.java`, `*Tests.java` and `*TestCase.java`, so `CalculatorControllerIT` was skipped without any warning.	Added explicit `<includes>` for `*Test.java` and `*IT.java` in `pom.xml`. The run reports 16 tests and two XML files.

10.3 Nginx kept old state after a redeploy

Symptom	Cause	Fix and verification
A redeploy replaces only the calculator container, because only its image tag changes.	Nginx reads its bind-mounted config and resolves the `calculator` host name only when it starts, so without a restart a changed `nginx.conf` (the demo change) never appears, and a new container IP could give 502 errors.	`depends_on.calculator.restart: true` makes Compose restart Nginx after it updates the app. Confirmed by build #3, which made the new landing page visible.

10.4 Maven used the wrong JDK in Jenkins

Symptom	Cause	Fix and verification
Maven builds failed with "release version 21 not supported".	The Mac's default JDK is 17, and Jenkins started by launchd gets a minimal PATH without the Homebrew tools.	Set the global Jenkins variables `JAVA_HOME` (JDK 21) and `PATH+EXTRA=/opt/homebrew/bin:/usr/local/bin`. The Jenkinsfile stays free of machine-specific paths.

10.5 Docker broken by Docker Desktop leftovers

Symptom	Cause	Fix and verification
Docker Desktop had been removed, but `~/.docker` still pointed to its credential helper and contained 14 dangling plugin links, including one for `docker compose`.	Any `docker login` or pull would fail looking for `docker-credential-desktop`, and the dead `docker-compose` link would shadow the working plugin.	Backed up `~/.docker`, removed the helper setting and the dead links, installed Colima with Homebrew's Compose and Buildx plugins, and pointed Docker at them with `cliPluginsExtraDirs`.

10.6 JVM memory flags were ignored

Symptom	Cause	Fix and verification
The Compose file set `JAVA_OPTS`, but the container started with default heap sizes.	The exec-form `ENTRYPOINT ["java", "-jar", ...]` never expands `JAVA_OPTS`; nothing reads that variable.	Switched to `JAVA_TOOL_OPTIONS`, which the JVM reads by itself; the log shows `Picked up JAVA_TOOL_OPTIONS: -Xms128m -Xmx384m`, and the entry point keeps its exec form.

10.7 Pre-demo checks always passed

Symptom	Cause	Fix and verification
The preflight script printed "All preflight checks passed" even when Docker and Jenkins were down.	With `set -e`, bash does not stop when the failing command is the left side of `check && echo OK`.	Put each check on its own line so the first failure stops the script; also check Jenkins at `/login`, because the home page answers 403 once security is on.

11. How to Run and Reproduce

Commands

```
# tests and the JAR (JDK 21 required)
mvn test
mvn clean package && java -jar target/calculator-devops-1.0.0.jar

# one container
colima start                      # or start Docker Desktop
docker build -t mayanktan/calculator-devops:1.0.0 .
docker run --rm -d -p 8090:8080 --name calc mayanktan/calculator-devops:1.0.0

# full stack behind Nginx
docker compose up -d --build
curl 'http://localhost/api/v1/add?a=2&b=3'

# pipeline: push to main; Jenkins (http://localhost:8080) builds and deploys within 2 minutes
./scripts/preflight.sh
```

The step-by-step version, with expected output for every command and a troubleshooting table, is in `test.pdf`.

12. Conclusion and Future Work

The pipeline meets every explicit requirement of the problem statement, and five real runs show it working: a push is built, tested, published and deployed in about 40 seconds without manual steps, and a failing test stops the pipeline before anything is published. Running the system end to end exposed problems a static review would have missed, most importantly the Nginx Host-header bug that would have failed every deployment.

The capstone will extend the same repository:

- **Ansible** to provision the build and deployment host instead of setting it up by hand.
- **Kubernetes** manifests (Deployment, Service, Ingress) using the Actuator liveness and readiness probes that are already enabled.
- **Prometheus and Grafana** on Spring Boot Actuator metrics for monitoring and dashboards.
- A **secured Jenkins** (login and an API token) so a GitHub webhook can replace polling.
- A **multi-architecture image** (`linux/amd64` and `linux/arm64`) built with `docker buildx`, since the current image is `arm64` only.

Appendix A. Screenshots

All screenshots were taken on 28-29 October 2026 from the running system. The first group is the set required by the problem statement (green pipeline run, email notification, docker ps, sample API responses); the second group is supporting evidence. Terminal figures show the exact output of the commands shown. The success emails for builds #1-#3 are also in `docs/screenshots/`.

Required screenshots

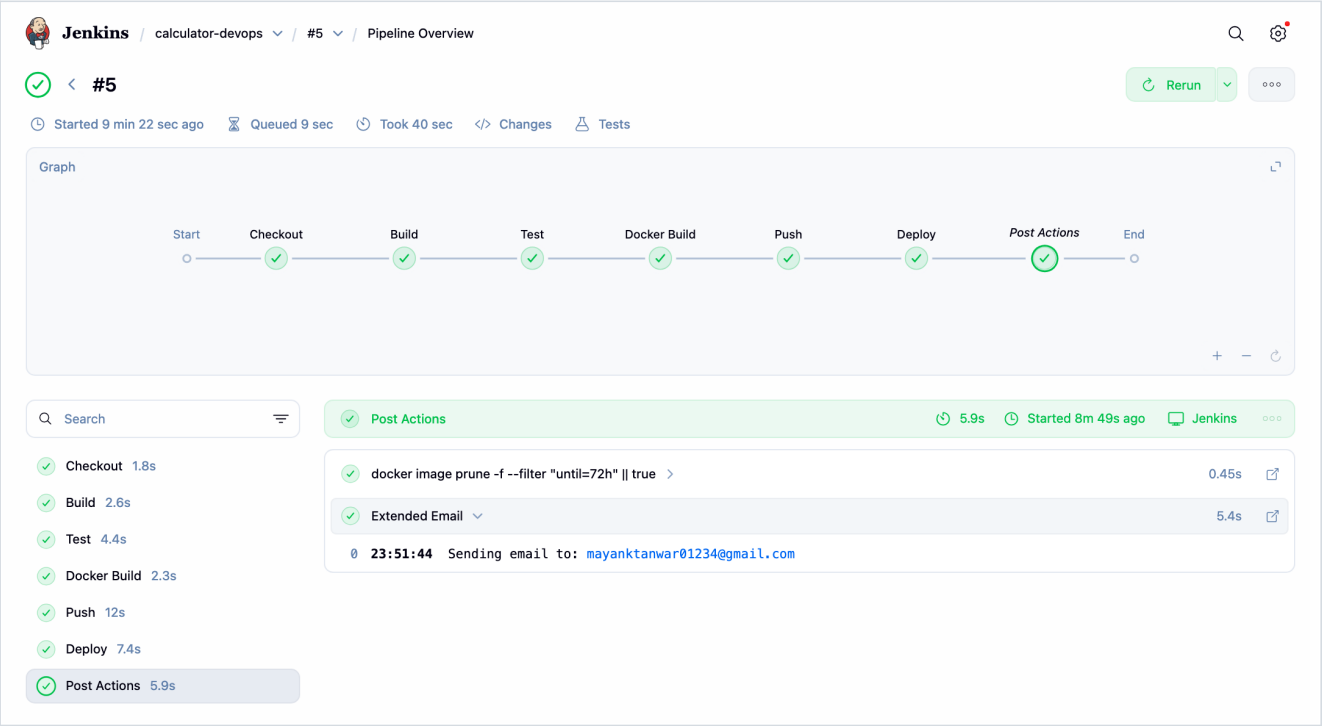


Figure A1. Green Jenkins pipeline run: build #5, all six stages passed; the post step sends the email.

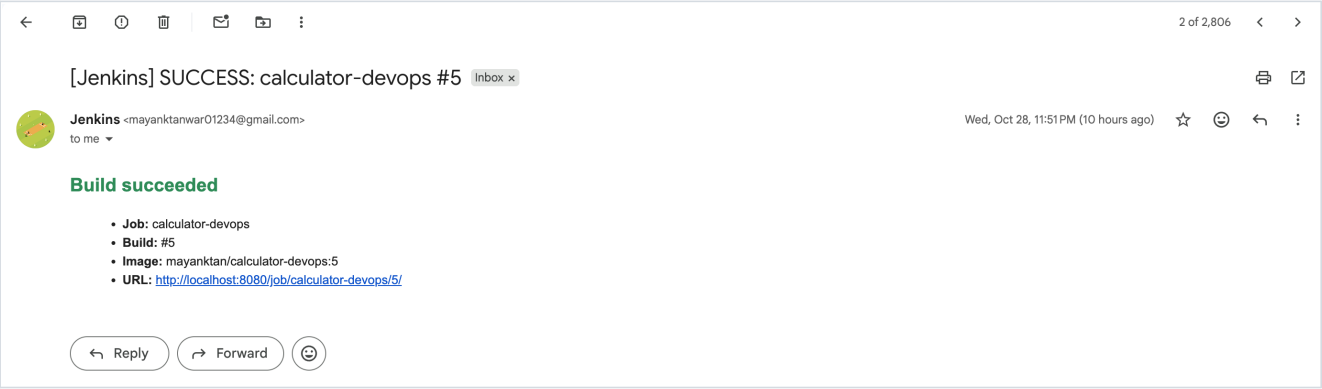


Figure A2. Email notification (success): build #5.

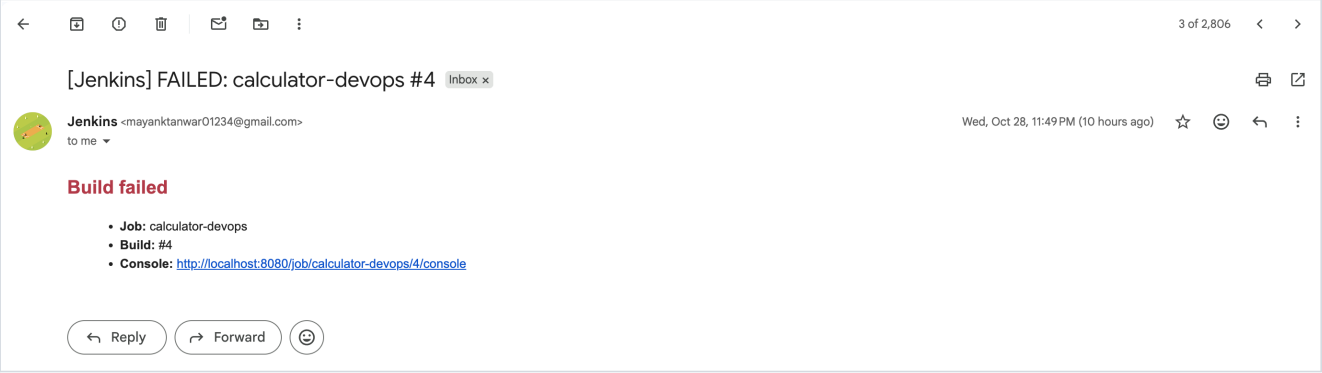


Figure A3. Email notification (failure): build #4, the deliberately broken test.

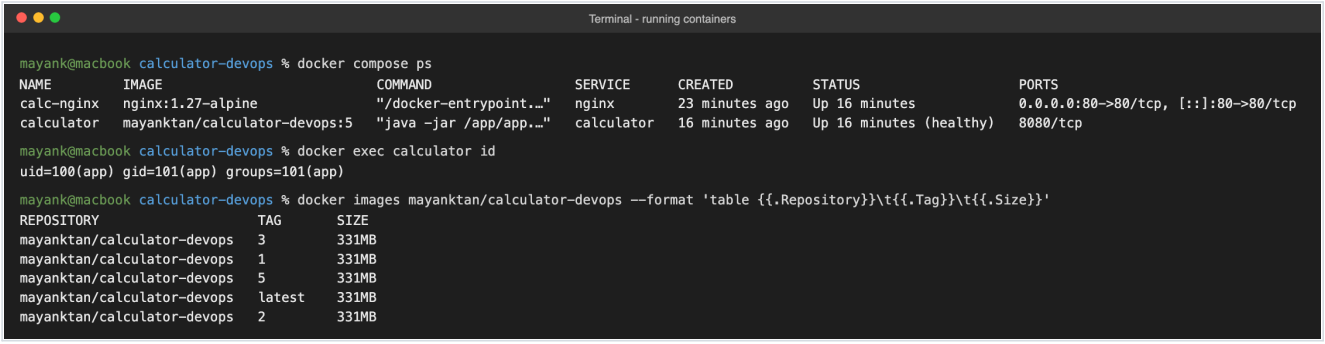


Figure A4. docker ps: `docker compose ps` shows calculator healthy and Nginx published on port 80; the app runs as uid 100 (app); one image tag per build.

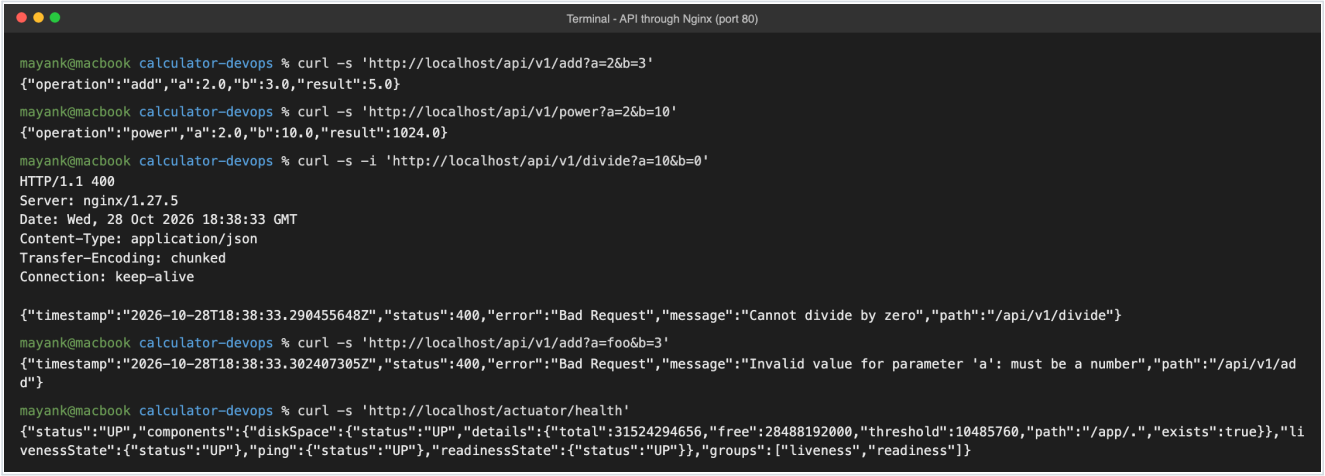


Figure A5. Sample API responses through Nginx on port 80, including the HTTP 400 errors and the health endpoint.

Supporting screenshots

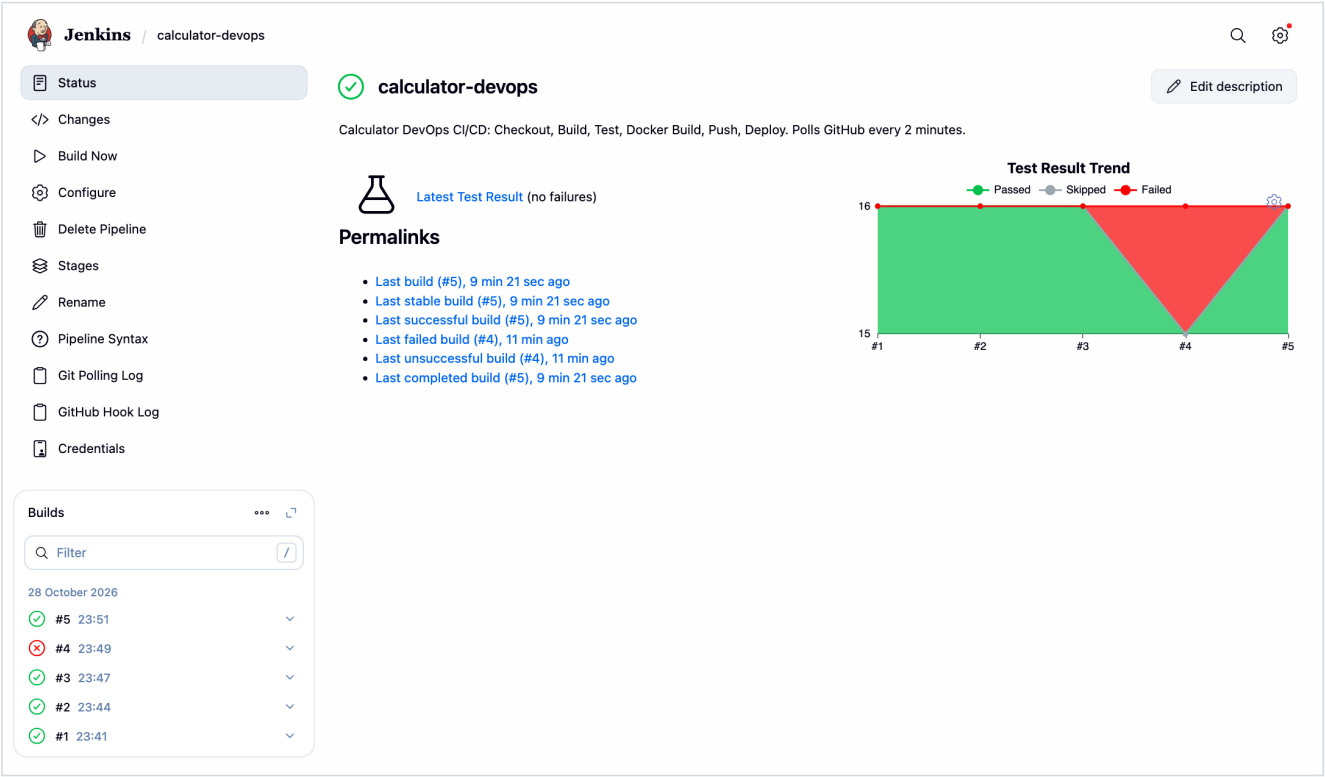


Figure A6. Jenkins job: builds #1-#5 (#4 failed on purpose) and the test result trend.

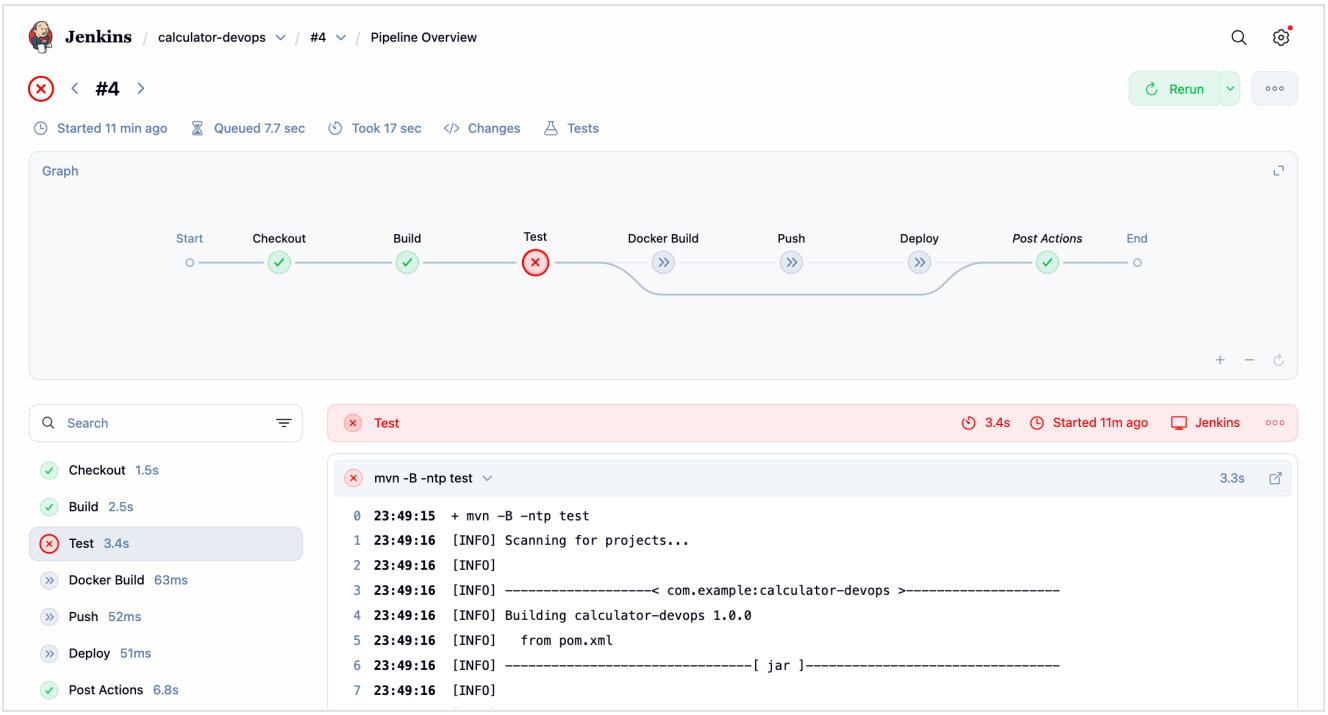


Figure A7. Build #4: Test failed, so Docker Build, Push and Deploy were skipped.

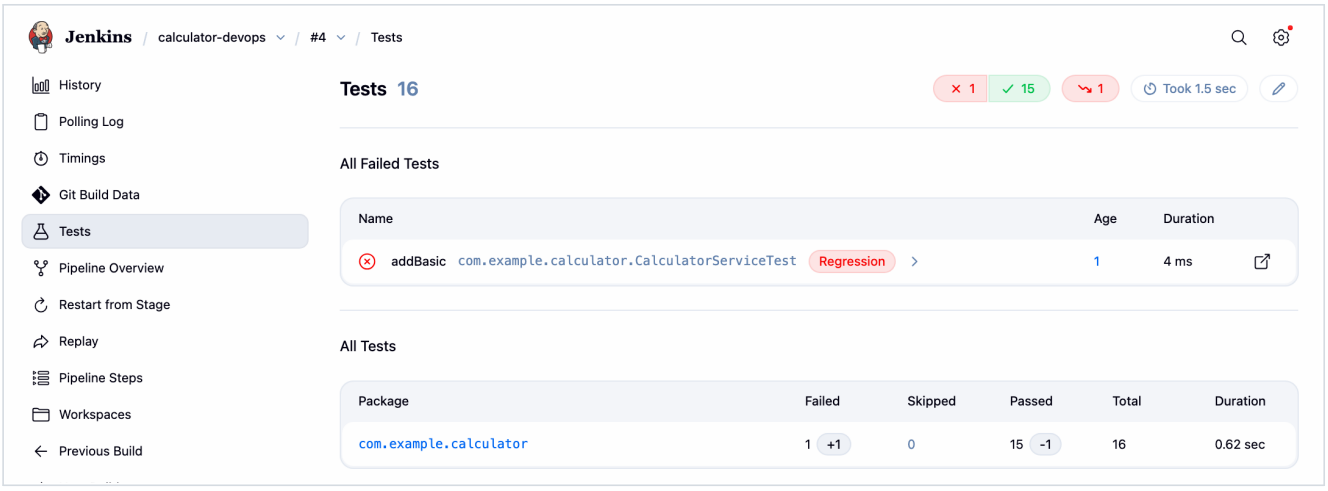


Figure A8. Build #4 test report: the deliberately broken `addBasic` test; 15 of 16 passed.

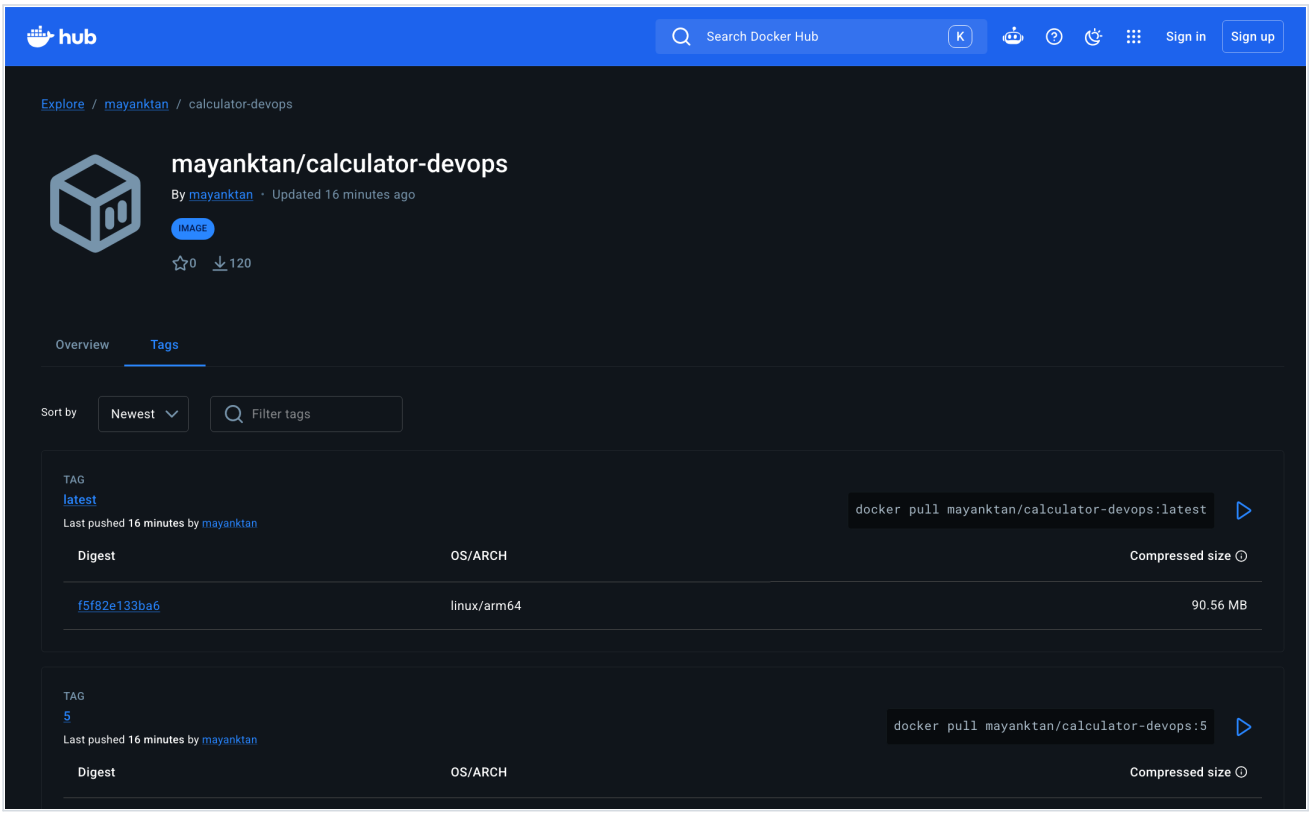


Figure A9. Docker Hub: public repository; `latest` and build 5 share one image digest.

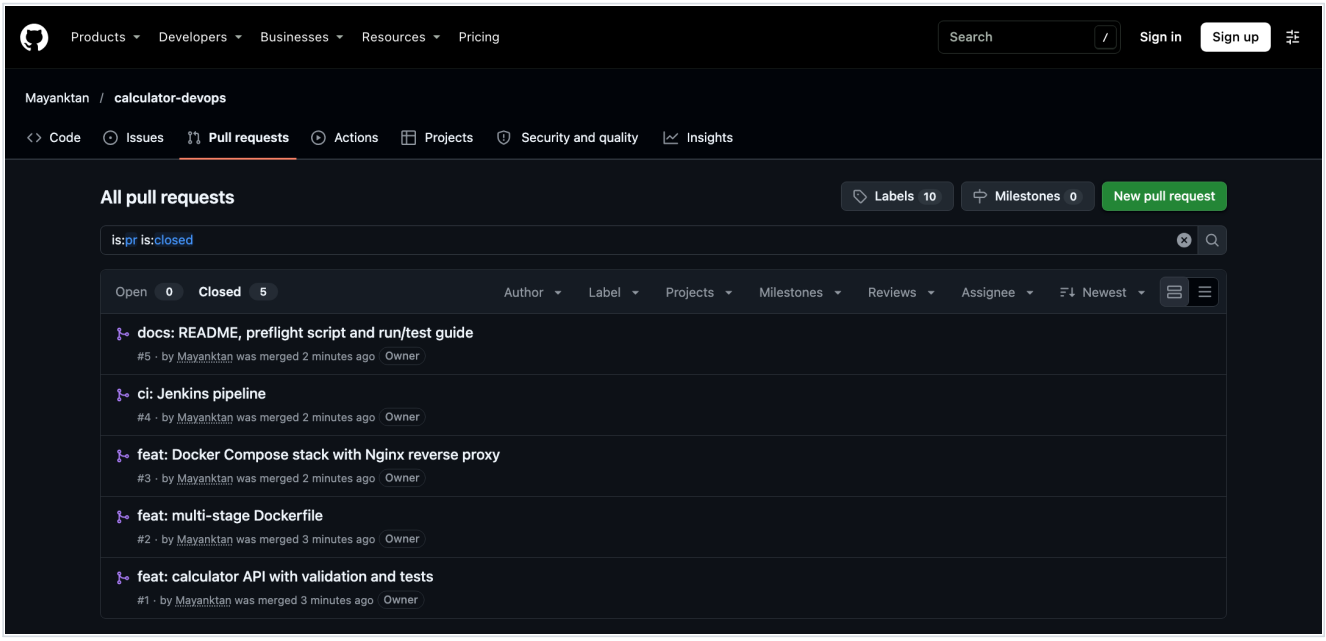


Figure A10. GitHub: the five merged pull requests.

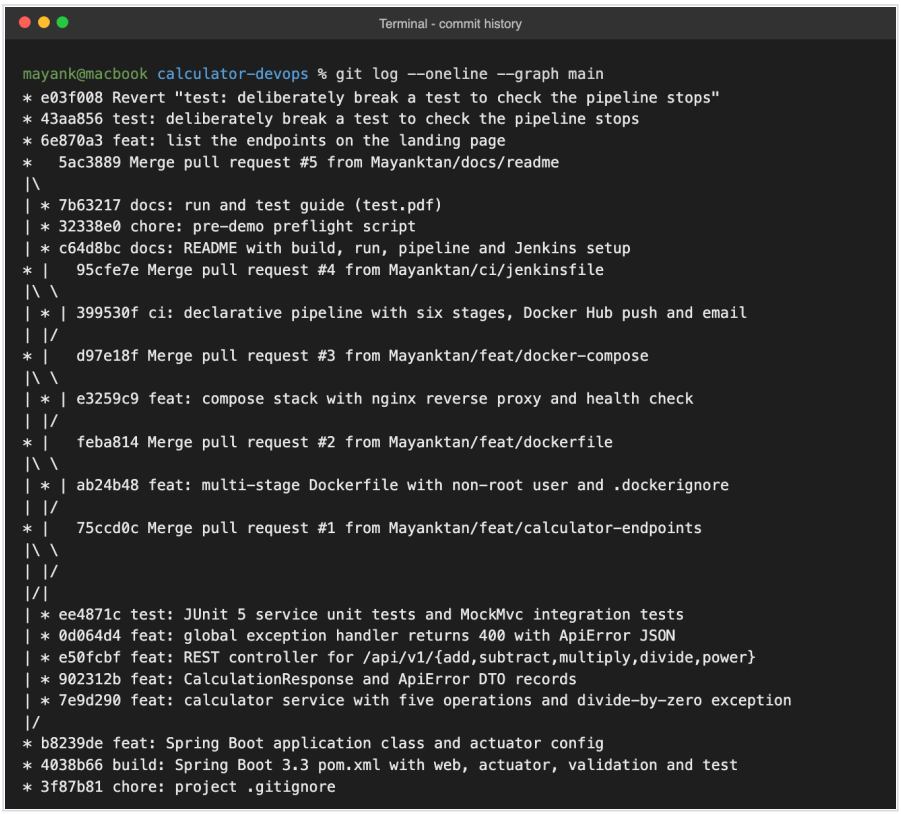


Figure A11. Commit history of `main` (`git log --graph`): five feature branches merged through pull requests, then the pipeline tests.

Appendix B. Versions

Tool or plugin	Version
Spring Boot / Java / Maven	3.3.4 / 21 / 3.9.11 (host), 3.9.9 (image)
Docker Engine / CLI / Compose / Buildx	29.5.2 / 29.9.2 / 5.6.1 / 0.38.1
Colima	0.10.3 (vz, virtiofs, 4 CPU, 4 GB)
Base images	maven:3.9.9-eclipse-temurin-21, eclipse-temurin:21-jre-alpine, nginx:1.27-alpine
Jenkins	LTS 2.528.2
Jenkins plugins	Pipeline 608.v67378e9d3db_1, Git 5.8.0, GitHub 1.45.0, Credentials Binding 702.vfe613e537e88, JUnit 1369.v15da_00283f06, Email Extension 1933.v45cec755423f, Timestamper 1.30, Pipeline Graph View 661.v6003f4542123